

Real-Time Embedded Systems (#41505)

Laboratory Module: Linux Processes, Signals and Threads

Paulo Pedreiras, pbrp@ua.pt
Pedro Fonseca, pf@ua.pt

Department of Electronics, Telecommunications and Informatics
University of Aveiro
Academic year 2026/2027

Laboratory Module: Linux Processes, Signals and Threads

Total work duration: 10 hours

Learning Outcomes:

- Process structure, type, and scheduling
- Starting new processes in different ways
- Parent, child, and zombie processes
- What signals are and how to use them
- Creating new threads within a process
- Synchronizing data access between threads in a single process
- Modifying the attributes of a thread
- Controlling one thread from another in the same process

Preparation: learn about relevant Linux commands and syscalls

- Study the notion of process, process structure, process table and attributes
- Investigate the following Linux commands and system calls:
  - ps, kill
  - system(), exec() and fork()
  - getpid(), getppid(), wait()/waitpid()
  - sigaction()
- Annex: syscall quick reference briefly summarizes the main syscalls required to solve these exercises

Part 1: Run a program from inside another program with system()

- Study the documentation of the “system()” library function. E.g. “$man 3 system”

Exercise 1.1:

Write a program called “myPS” that executes the command “ps” with flags “aux”.

Exercise 1.2:

Write a program that takes as input a text filename and prints the number of characters and words it contains.

Hint: investigate the Linux command “wc”.

Exercise 1.3:

Write program that takes as inputs a string and a number (n) between 1 and 99. The program should create folders named with the concatenation of the string and the numbers 1, 2, … n. E.g., “$createFold Group 10” will create folders “Group1”, “Group2”, … “Group10”.

Hint: investigate the Linux command “mkdir”.

Exercise 1.4:

Write a program as in Exercise 1.3, but that removes folders that follow a pattern, instead of creating them. E.g., “$deleteFold Group 10” will remove folders “Group1”, “Group2”, … “Group10”.

Hint: investigate the Linux command “rmdir”. Note that folders may not be empty, thus it may be necessary to use some special parameter to rmdir

Suggestion: modify the program to support an option to print the name of each folder and ask the user to confirm if she/he really wants to remove it (interactive removal)

Part 2: Run a program from inside another program with the “exec” family of functions

- Study the documentation of the “exec” family of functions (e.g. “$man 3 exec”)

Exercises 2.1, 2.2 and 2.3:

Repeat exercises 1.1, 1.2, 1.3 and 1.4 using one of the “exec” family of functions

Part 3: creating separate processes - fork

- Study the documentation of the “fork” system call. Check also functions “getpid()”, “getppid()” and “wait()”/”waitpid().

Exercise 3.1:

Create a simple program that forks a child process. Both parent and child processes print a message indicating who they are, its own pid and the pid of the parent/child. At close they should also print a message indicating they are are finishing (e.g. “Process PID 12345 is finishing now”).

Exercise 3.2:

Modify the previous example and make the child process do some computation that takes a significant amount of time (or simply put a delay to emulate the time a process takes to do work) and check the sequence in which the processes start and terminate.

Modify the program to make sure that the parent process only closes after the child one.

Note: be careful with buffering on stdout. Terminate all "printf()" with a "\n" otherwise message order will be altered. It is also advisable to use "fflush(stdout)" at critical points.

Part 4: Signals

- Study the documentation of sigaction
- Note that signals can also be handled by the signal() syscall. This syscall is still found in old programs but is deprecated. sigaction() has a more cleanly defined and reliable interface, thus it should be used in all new programs

Exercise 4.1:

Create a program that catches signals SIGINT, SIGHUP, SIGUSR1 and SIGCONT and prints a message identifying the signal, when one is received. Use the sigaction() syscall.

Note: signals can be sent form the command line with the command "$kill -SIG PID". E.g. "$kill -USR1 44799" sends signal SIGUSR1 to process with PID 44799. Moreover, hitting "CTRL+C" on a terminal were a process was launched sends a SIGINT to the running process.

Note also that "$kill -KILL PID" terminates a process.

Exercise 4.2:

Create a program that counts seconds since its start and prints the elapsed number of seconds in the terminal (i.e. a timer application). The counting can be suspended, reset and resumed with the following signals:

- SIGINT – Reset and continue (resumes from 0)
- SIGHUP – Suspends the counting
- SIGCONT – Resumes the counting.

Base your solution in the sigaction() syscall.

Exercise 4.3:

The previous program terminates if you send a signal that that is not caught. E.g. try running the program and send a USR1 signal ($kill -USR1 PID). Usually this is not the desired behavior, and in these cases unwanted signals should be ignored, not interfering with normal program execution. Modify the program so that signals other than SIGINT, SIGHUP and SIGCONT are ignored. Base your solution in the sigaction() syscall and its mask argument. Note that some signals cannot be caught, blocked, or ignored (SIGKILL, SIGSTOP).

Exercise 4.4:

Create a process that forks. The parent process shall send ten times the SIG_USR1 signal to the child process, one per second, and then terminates. The child process should also terminate after receiving the ten SIG_USR1 signals. Moreover, both process should print a message when send/receive a signal.

Hint: check the kill(pid_t pid, int sig) syscall

Exercise 4.5:

The sa_mask field of the sigaction struct specifies a set of signals to be added to the process's signal mask before the sa_handler function is called. These are the set of signals that are blocked and won't be delivered to the process. This is often necessary to prevent situations where a signal is received before its handler has run to completion, eventually causing a race condition.

To learn how to use this mechanisms, create a program in which SIGUSR1 increments a counter. While inside the SIGUSR1 handler, SIGUSR2 shall be blocked temporarily, using the signal mask. Thus, if SIGUSR2 arrives during the execution of the SIGUSR1 handler, its delivery is delayed and only after SIGUSR handler terminates the SIGUSR2 executes.

The amount of processing, and thus the time required, by the SIGUSR1 is minimal, so it is very difficult to send a SIGUSR2 signal during the SIGUSR1 execution. To make this feasible from the command line just add a delay inside the handler (e.g. 5s). It can also be useful to print messages that show the user when the handler is execution. E.g.:

```
void handle_sigusr1(int sig) {
    (void)sig;
    printf("[Handler] Entered SIGUSR1 handler\n");

    // simulate long work inside handler
    sleep(5);

    counter++;
    printf("[Handler] Leaving SIGUSR1 handler, counter=%d\n",
counter);
    fflush(stdout);
}
```

Note: remember that you can run a process in background by adding a "&" at the end of the command line. E.g. "$ex4_5&" runs "ex4_5" in background, so you can use the same terminal to send signals with the kill command. In many cases, like this one, this facilitates testing.

Part 5: POSIX Threads

- Review the concept of threads
- Study the documentation related with IEEE POSIX threads, specially the syscalls pthread_create(), pthread_exit() and pthread_join()

Exercise 5.1

Create a program that spawns two threads. Each thread prints a message identifying it at beginning (e.g. “Thread 1 starting”), then waits a few seconds and finishes. Before finishing, it should also print a descriptive message. The main thread should wait for both threads to finish, and then finishes itself, also printing a descriptive message.

Example of the intended output:

Main thread: creating two threads named 1 and 2

Thread 2 starting

Thread 1 starting

Thread 1 finishing

Thread 2 finishing

Main thread: both threads finished, exiting.

Note: when compiling programs using threads you should use the “-pthread” arg to gcc. E.g. “$gcc threads.c -o threads -pthread”. This arg makes gcc add necessary compile and link flags. In older systems it was necessary to have more flags and defines. A typical compile line, still found in many books and legacy code is “$cc -D_REENTRANT -I/usr/include/nptl thread1.c –o thread1 -L/usr/lib/nptl -lpthread”. From some years NPTL is the default thread library, so the “-I” and “-L” are not necessary. Also “-pthread” already defines “-D_REENTRANT”, so you get the simpler line above.

Exercise 5.2

Threads of the same process share the address space, so they can communicate via shared variables. To verify that, create a program that spawns two threads. Make one of the threads update one counter variable every second, and the other one print the contents of the variable whenever it

changes. Use a flag and polling to alow the print thread to detect updates to the counter variable. The process should terminate after a given number of executions, defined at compile time.

Exercise 5.3

Threads can also handle signals. Create a program that creates two threads and wait for the reception of a signal. When a signal is received, they print a suitable message and terminate. The main thread creates both threads and then sends signal SIG_USR1 to each one, with a one second interval. Then it terminates, after being sure that both threads had terminated.

Hint: check syscalls pthread_kill() and pause().

Annex: syscall quick reference

1. Process Management

`system(const char *command)`

- **What it does**: Runs a shell command from inside a program.
- **Use**: Simple way to run external commands (`ps`, `wc`, `mkdir`, `rmdir`, etc.) from C.
- **Syntax**:

```
int ret = system("ls -l");
```

- **Notes**: Blocks until the command finishes. Returns the command's exit status.

`exec()` family (`execl`, `execp`, `execv`, etc.)

- **What it does**: Replaces the current process image with a new program.
- **Use**: To run another program *inside* the current process after a `fork()`.
- **Syntax** (example):

```
execl("/bin/ls", "ls", "-l", NULL);
```

- **Notes**: On success, does not return; the calling process becomes the new program.

`fork()`

- **What it does**: Creates a new process (child).
- **Use**: To demonstrate parent-child processes.
- **Syntax**:

```
pid_t pid = fork();
```

- **Notes**: Returns `0` in the child, child's PID in the parent, `-1` on error.

`getpid()` and `getppid()`

- **What they do**: Return the PID of the calling process and its parent process.
- **Use**: Identify parent/child relationships.

- **Syntax**:

```
pid_t pid = getpid();
pid_t ppid = getppid();
```

`wait()` / `waitpid()`

- **What they do**: Make the parent process wait for a child to finish.
- **Use**: Prevents zombies and enforces execution order.
- **Syntax**:

```
int status;
pid_t cpid = wait(&status);
```

---

2. Signals

`sigaction(int signum, const struct sigaction *act, struct sigaction *oldact)`

- **What it does**: Installs a signal handler.
- **Use**: Reliable way to catch, ignore, or handle signals (`SIGINT`, `SIGHUP`, etc.).
- **Syntax**:

```
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);
```

`kill(pid_t pid, int sig)`

- **What it does**: Sends a signal to a process (or process group).
- **Use**: For inter-process signaling (`SIGUSR1`, `SIGKILL`, etc.).
- **Syntax**:

```
kill(child_pid, SIGUSR1);
```

`pause()`

- **What it does**: Suspends execution until a signal is received.
- **Use**: For threads or processes to wait passively for signals.

`pthread_kill(pthread_t thread, int sig)`

- **What it does**: Sends a signal to a specific thread in the same process.
- **Use**: Thread-level signal delivery.

---

3. POSIX Threads

`pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg)`

- **What it does**: Creates a new thread of execution in the same process.
- **Use**: Parallel execution inside one process.

`pthread_exit(void *retval)`

- **What it does**: Terminates the calling thread.
- **Use**: Clean thread exit with optional return value.

`pthread_join(pthread_t thread, void **retval)`

- **What it does**: Waits for another thread to finish.
- **Use**: Synchronizes threads, like `wait()` for processes.

`pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset)`

- **What it does**: Change or examine the calling thread's signal mask.
- **Use**: Control which thread receives signals.

Summary

- **Process creation & management**: `system()`, `exec*()`, `fork()`, `getpid()`, `getppid()`, `wait()`.
- **Signals**: `sigaction()`, `kill()`, `pause()`, `pthread_kill()`.
- **Threads**: `pthread_create()`, `pthread_exit()`, `pthread_join()`, `pthread_sigmask()`.